



PROJE ANALİZ RAPORU

Ders: BİL204 / Yazılım Mühendisliği

Bölüm/Şube: Bilgisayar Mühendisliği

Grup Adı: Denem

Proje Adı: Proje OBS

Teslim Tarihi: 16.03.2026

Grup Üyeleri:

- Z. Esat Saygın (25100011473)
- Yakup Çaktı (25100011018)
- Muhammed Şerbetçioğlu (24100011080)
- Yaşar A. Yarayan (24100011016)
- Muhammed Barbin (24100011056)
- Tahir Allahverdiyev (24100011822)

İçindekiler

İçindekiler	2
1. Giriş	3
2. Genel Bakış	3
3. Önerilen Sistem	3
3.1 İşlevsel Gereksinimler (Functional Requirements)	4
3.2 İşlevsel Olmayan Gereksinimler (Non-Functional Requirements)	6
3.3 Sözde Gereksinimler (Pseudo Requirements)	7
4 Sistem Modelleri	8
4.1 Kullanım Durumu (Use Case) Modeli	8
4.2 Sınıf Diyagramı	12
4.3 Durum Diyagramı	14
4.4 Aktivite Diyagramı	14
4.5 Sıra (Sequence) Diyagramı	14
4.6 Kullanıcı Arayüzü	14
5. Referanslar	14

1. Giriş

Bu proje, üniversite öğrencilerinin ve akademisyenlerin akademik süreçlerini dijital ortamda yönetmesini kolaylaştıran bir Öğrenci Bilgi Sistemi (OBS) geliştirmeyi amaçlamaktadır.

Sistem, ders kaydı, not giriş/görüntüleme, transkript görüntüleme işlemlerini desteklemektedir. Hedef kullanıcılar üniversite öğrencileri, akademisyenler ve idari personeldir.

Rapor beş bölümden oluşmaktadır. Birinci bölümde projenin amacı ve kapsamı açıklanmıştır. İkinci bölümde sistemin genel tanımı verilmiştir. Üçüncü bölümde gereksinimler belirtilmiş olup dördüncü bölümde sistem modelleri sunulmuştur. Son bölümde ise referanslar verilmiştir.

2. Genel Bakış

Sistem, üniversite öğrencilerinin ders kayıtlarını yapabilmesi, notlarını ve transkriptlerini görüntülemesi; akademisyenlerin öğrencilerin ders kayıt işlemlerini onaylaması, notlarını girmesi ve görebilmesi; idari personelin admin panelinden kullanıcıları yönetebilmesi işlevlerini sağlamaktadır. Öğrencilerin kayıt işlemi excel dosyasından isim, soyisim, bölüm gibi bilgilerin çekilip öğrenci numarası oluşturularak veri tabanına kaydedilmesi şeklinde olacaktır. Akademisyen ve idari personelin kaydı ise admin panelinden manuel şekilde yapılacaktır.

3. Önerilen Sistem

Bu bölümde önerilen Öğrenci Bilgi Sistemi'nin (OBS) genel tanımı, işlevsel ve işlevsel olmayan gereksinimleri detaylı olarak açıklanmaktadır.

Sistem, web tabanlı bir uygulama olarak geliştirilecek ve üniversite öğrencilerinin ders kayıt süreçlerini, akademisyenlerin not giriş işlemlerini ve idari personelin kullanıcı yönetimi görevlerini dijital ortamda gerçekleştirmesine olanak sağlayacaktır.

Hedef Kullanıcılar: Öğrenciler, Akademisyenler ve Yönetici (İdari Personel)

Temel Özellikler: Ders kayıt yönetimi, not giriş ve görüntüleme, transkript görüntüleme ve indirme, rol tabanlı yetkilendirme

3.1 İşlevsel Gereksinimler (Functional Requirements)

FR-1: Kimlik Doğrulama ve Yetkilendirme

FR-1.1: Sistem, kullanıcıların kullanıcı adı ve şifre ile giriş yapmasını sağlamalıdır.

FR-1.2: Kimlik doğrulama JWT (JSON Web Token) tabanlı olmalıdır.

FR-1.3: Sistem üç farklı kullanıcı rolünü desteklemelidir: Öğrenci, Akademisyen ve Yönetici.

FR-1.4: Her kullanıcı rolü için farklı yetki seviyeleri tanımlanmalıdır.

FR-1.5: Kullanıcılar sistemden güvenli bir şekilde çıkış yapabilmelidir.

FR-2: Öğrenci İşlevleri

FR-2.1 Ders Kaydı: Öğrenciler, sistemde mevcut olan derslere kayıt yapabilmelidir. Ders kaydı akademisyen onayına sunulmalıdır.

FR-2.2 Ders ve Not Görüntüleme: Öğrenciler, kayıtlı oldukları dersleri ve bu derslere ait notlarını (vize, final, ortalama, harf notu) görüntüleyebilmelidir.

FR-2.3 Transkript Görüntüleme: Öğrenciler, tüm dönemlere ait ders notlarını ve genel not ortalamalarını içeren transkriptlerini görüntüleyebilmelidir.

FR-2.4 Transkript İndirme: Öğrenciler, transkriptlerini PDF formatında indirebilmelidir.

FR-2.5 Yetkilendirme: Öğrenciler yalnızca kendi bilgilerine erişebilmelidir.

FR-3: Akademisyen İşlevleri

FR-3.1 Ders Görüntüleme: Akademisyenler, verdikleri dersleri listeleyebilmelidir.

FR-3.2 Öğrenci Listesi: Akademisyenler, verdikleri derslere kayıtlı öğrencileri görüntüleyebilmelidir.

FR-3.3 Not Giriş ve Güncelleme: Akademisyenler, sorumlu oldukları derslerin öğrencilerine ait vize ve final notlarını girebilmeli ve güncelleyebilmelidir.

FR-3.4 Ders Kaydı Onaylama: Akademisyenler, verdikleri derslere yapılan öğrenci kayıt taleplerini onaylayabilmelidir.

FR-3.5 Yetkilendirme: Akademisyenler yalnızca kendi derslerine ait bilgilere erişebilmelidir.

FR-4: Yönetici İşlevleri

FR-4.1 Toplu Öğrenci Kaydı: Yönetici, Excel dosyası yükleyerek toplu öğrenci kaydı yapabilmelidir. Sistem, Excel dosyasından alınan bilgilerle (ad, soyad, bölüm) otomatik olarak öğrenci numarası oluşturmalı ve veritabanına kaydetmelidir.

FR-4.2 Manuel Akademisyen Kaydı: Yönetici, admin panelinden akademisyen kaydı yapabilmelidir.

FR-4.3 Ders Yönetimi: Yönetici, ders oluşturabilir, mevcut dersleri güncelleyebilir ve silebilmelidir.

FR-4.4 Kullanıcı Yönetimi: Yönetici, tüm kullanıcıları listeleyebilmeli ve yönetebilmelidir.

FR-5: Veri İşleme

FR-5.1 Not Hesaplama: Sistem, girilen vize ve final notlarından otomatik olarak ders ortalamasını hesaplamalıdır. Hesaplama formülü: $\text{Ortalama} = (\text{Vize} \times 0.4) + (\text{Final} \times 0.6)$

FR-5.2 Harf Notu Belirleme: Sistem, hesaplanan ortalamaya göre otomatik olarak harf notu atamalıdır.

FR-5.3 Öğrenci Numarası Üretimi: Sistem, toplu kayıt sırasında benzersiz öğrenci numarası otomatik olarak oluşturmalıdır.

3.2 İşlevsel Olmayan Gereksinimler (Non-Functional Requirements)

NFR-1: Güvenlik

NFR-1.1 Kimlik Doğrulama: Tüm API endpoint'leri, giriş yap ve benzeri açık endpoint'ler hariç, kimlik doğrulama (JWT token) gerektirmelidir.

NFR-1.2 Şifre Güvenliği: Kullanıcı şifreleri Django'nun varsayılan hash mekanizması kullanılarak güvenli bir şekilde saklanmalıdır.

NFR-1.3 Rol Tabanlı Erişim: Sistem, rol tabanlı yetkilendirme (Role-Based Access Control) uygulamalıdır. Kullanıcılar yalnızca yetkili oldukları kaynaklara erişebilmelidir.

NFR-2: Performans

NFR-2.1 Yanıt Süresi: API endpoint'leri normal koşullarda 1 saniyenin altında yanıt vermelidir.

NFR-2.2 Sayfalama: Listeleme işlemlerinde (öğrenci listesi, ders listesi vb.) sayfalama (pagination) kullanılmalıdır. Varsayılan sayfa boyutu 20 kayıt olmalıdır.

NFR-2.3 Eşzamanlılık: Sistem, normal kullanım koşullarında birden fazla kullanıcının eşzamanlı işlemlerini sorunsuz bir şekilde yönetebilmelidir.

NFR-3: Kullanılabilirlik

NFR-3.1 Arayüz Uyumluluğu: Web arayüzü responsive tasarım ilkelerine uygun olarak mobil ve masaüstü cihazlarda sorunsuz çalışmalıdır.

NFR-3.2 Kullanıcı Deneyimi: Arayüz, kullanıcı dostu ve sezgisel olmalıdır.

NFR-3.3 Hata Yönetimi: Sistem, kullanıcılara anlaşılır hata mesajları göstermelidir. Hata mesajları Türkçe olmalıdır.

NFR-3.4 API Standardı: Backend API, REST (Representational State Transfer) standartlarına uygun olmalıdır.

NFR-4: Bakım Yapılabilirlik

NFR-4.1 Modüler Yapı: Kod, Django apps yapısı kullanılarak modüler bir şekilde organize edilmelidir.

NFR-4.2 Dokümantasyon: API endpoint'leri dokümanite edilmelidir.

NFR-4.3 Versiyon Kontrolü: Proje kaynak kodu Git versiyon kontrol sistemi ile yönetilmelidir.

NFR-4.4 Genişletilebilirlik: Sistem, gelecekte yeni özellikler eklenmesine imkan verecek şekilde tasarlanmalıdır.

NFR-5: Uyumluluk

NFR-5.1 Tarayıcı Desteği: Web arayüzü, güncel tarayıcılarda (Chrome, Firefox, Safari, Edge) sorunsuz çalışmalıdır.

NFR-5.2 Veri Formatı: API, veri alışverişinde JSON formatını kullanmalıdır.

NFR-5.3 Dil: Sistem arayüzü ve hata mesajları Türkçe olmalıdır.

3.3 Sözde Gereksinimler (Pseudo Requirements)

PR-1 Platform: Sistem web tabanlı bir uygulama olarak geliştirilecektir.

PR-2 Backend Teknolojisi: Backend, Python 3.10+ ve Django kullanılarak Django REST Framework ile geliştirilecektir.

PR-3 Frontend Teknolojisi: Frontend, React framework'ü kullanılarak geliştirilecektir.

PR-4 Veritabanı: Prototip bir proje olduğundan ötürü SQLite kullanılacaktır.

PR-5 Kimlik Doğrulama Mekanizması: JWT (JSON Web Token) tabanlı kimlik doğrulama kullanılacaktır.

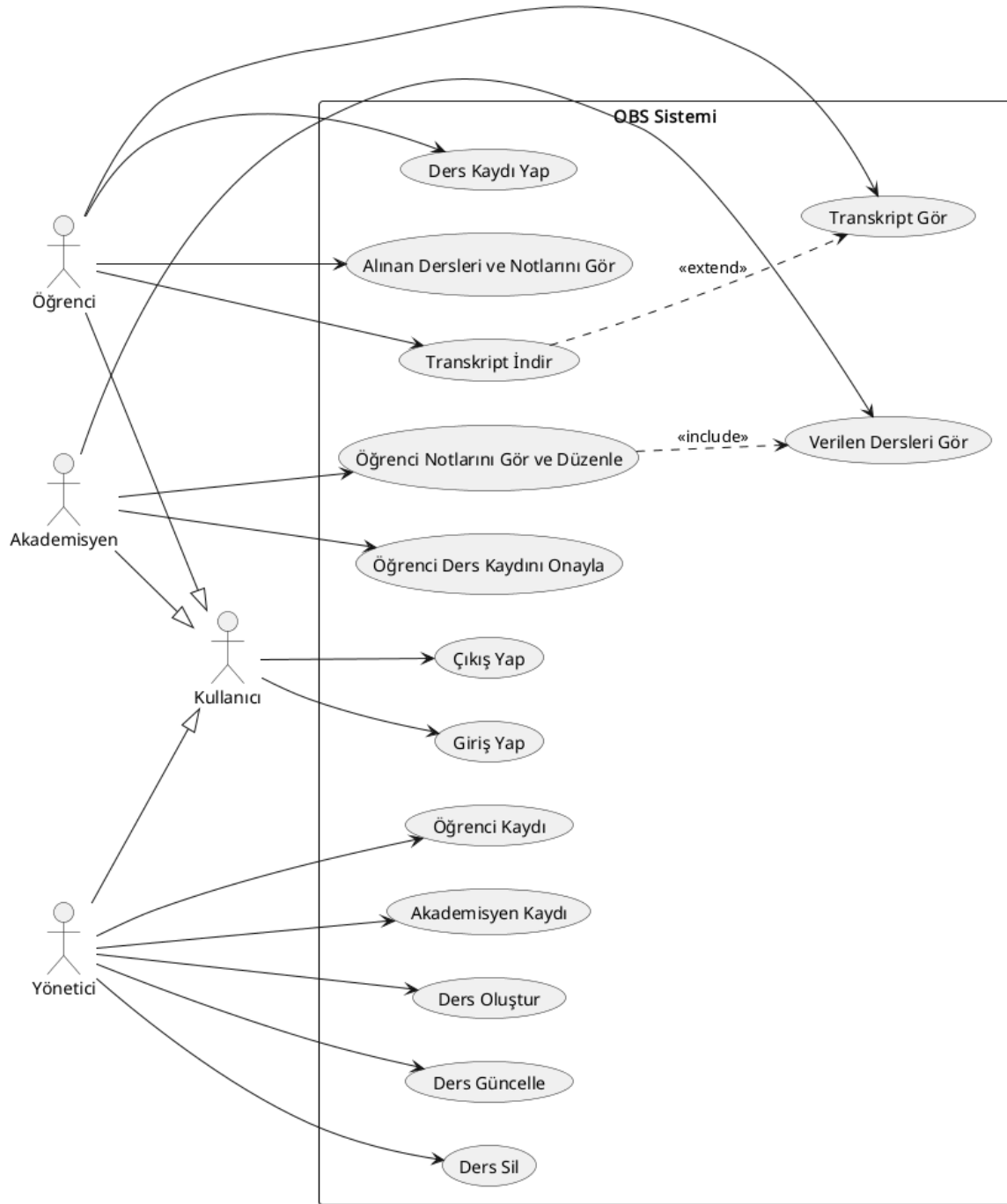
PR-6 Versiyon Kontrolü: Proje, Git versiyon kontrol sistemi kullanılarak GitHub üzerinde yönetilecektir.

PR-7 Deployment: Uygulama, web sunucusunda deploy edilebilir durumda olmalıdır.

4 Sistem Modelleri

Bu bölümde UML diyagramları ve modeller yer alır.

4.1 Kullanım Durumu (Use Case) Modeli



UC-1: Giriş Yap

Aktörler: Kullanıcı

Olay Akışı:

1. Kullanıcı web sitesini açar ve açılan ekranda hangi rolde giriş yapmak istediğini seçer.
2. Gerekli bilgi ve parolasını girmesi istenir. Gerekli bilgi olarak öğrenci rolü için öğrenci numarası, akademisyen ve yönetici rolleri için kullanıcı adı girilmesi istenir.

A1: Kullanıcı giriş yaptıktan sonra istediği zaman üst bardaki “Çıkış Yap” butonunu kullanarak güvenli çıkış yapabilir.

E1: Kullanıcı yanlış bilgi girerse sistem kullanıcıya yanlış bilgi girdiğini belirtir.

UC-2: Ders Kaydı Yap

Aktörler: Öğrenci

Olay Akışı:

1. Sistem öğrencinin sınıfına ve geçtiği veya kaldığı derslere göre alabileceği dersleri “Ders Kayıt” sayfasında görüntüler.
2. Öğrenci derslerin kontenjanı ve kendi ders alma limitine göre dersleri seçer ve onaylar.
3. Öğrencinin seçtiği dersleri veren akademisyenlerin kaydı onayına gönderilir.
4. Akademisyen onay verdiğinde öğrencinin ders kaydı tamamlanır.

E1: Öğrenci alabileceğinden daha fazla ders seçerse sistem buna izin vermez ve uyarı gösterir.

UC-3: Alınan Dersleri Ve Notlarını Gör

Aktörler: Öğrenci

Olay Akışı:

1. Sistem öğrenciye aldığı dersleri listeler ve notlarını, ortalamasını ve harf notunu gösterir.

UC-4: Transkript Gör

Aktörler: Öğrenci

Olay Akışı:

1. Öğrenci transkript görüntüleme işlemini başlatır.
2. Sistem öğrenci bilgilerinden hareketle bir transkript oluşturur ve öğrenciye gösterir.

A1: Öğrenci gösterilen transkripti *pdf* uzantısı ile indirebilir.

UC-5: Verilen Dersleri Gör

Aktörler: Akademisyen

Olay Akışı:

1. Sistem akademisyene, verdiği dersleri listeler.
2. Akademisyen derslerden birini seçer ve o dersi alan öğrencileri ve notlarını görür.

UC-6: Öğrenci Notlarını Gör Ve Düzenle

Aktörler: Akademisyen

Olay Akışı:

1. Akademisyen verdiği derslerden birini seçer ve o dersi alan öğrencileri görüntüler.
2. Öğrencilerin notlarını düzenler.
3. Sistem vize ve final notları girildiğinde ortalama ve harf notu hesaplar.
4. Sistem hesaplanan değerleri gösterir.

UC-7: Öğrenci Kaydı

Aktörler: Yönetici

Olay Akışı:

1. Yönetici öğrenci bilgilerini içeren Excel dosyasını sisteme yükler.
2. Sistem Excel dosyasından öğrencileri çıkarır ve her biri için özel birer öğrenci numarası oluşturur ve kaydeder.

A1: Yönetici öğrencileri manuel olarak da kaydedebilir.

UC-8: Akademisyen Kaydı

Aktörler: Yönetici

Olay Akışı:

1. Yönetici akademisyen kaydını başlatır.
2. Yönetici akademisyen bilgilerini sisteme girer ve kaydeder.

UC-9: Ders Oluştur

Aktörler: Yönetici

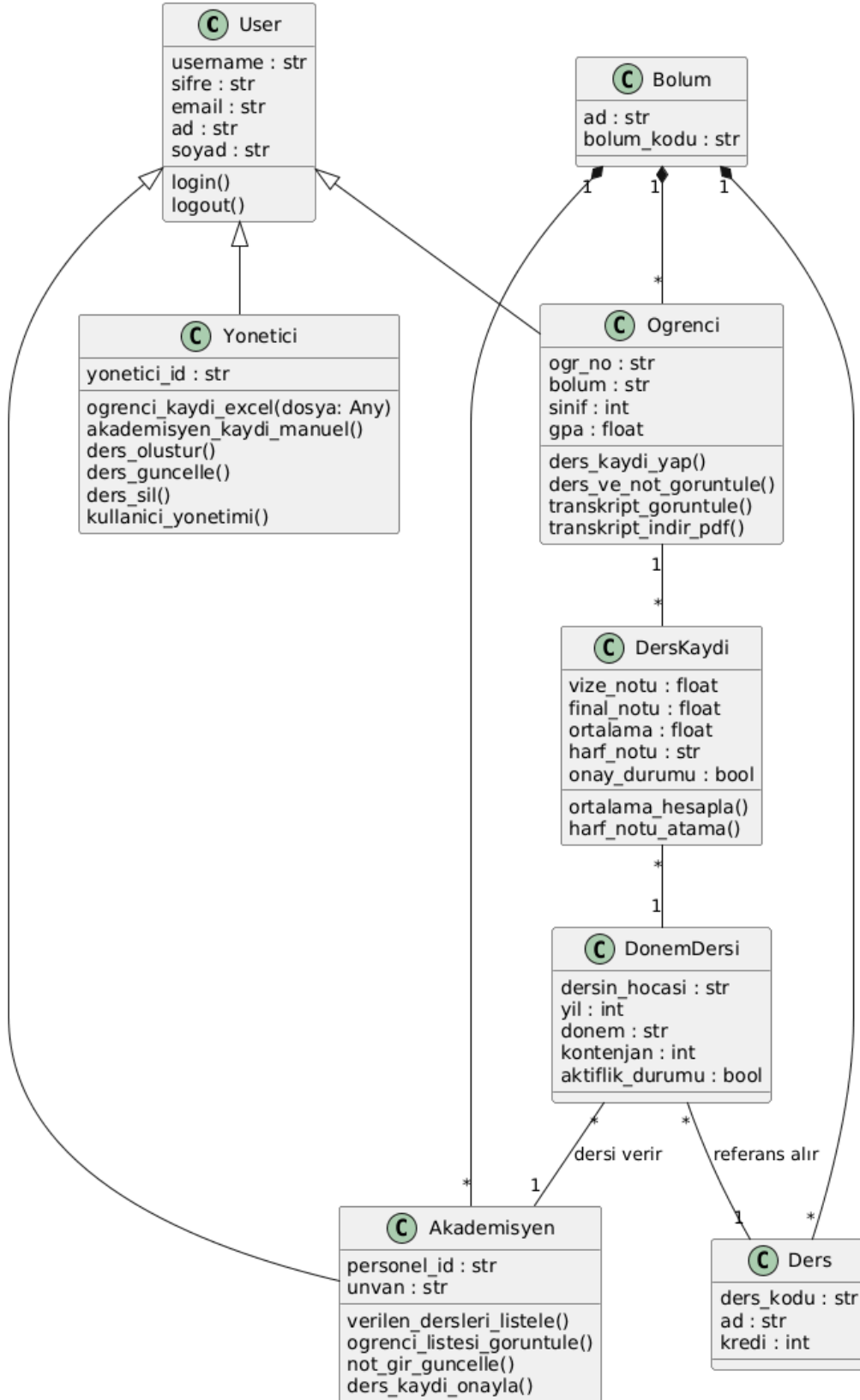
Olay Akışı:

1. Yönetici ders oluşturmaya başlatır.
2. Ders bilgilerini sisteme kaydeder.

A1: Yönetici ders bilgilerini güncelleyebilir.

A2: Yönetici dersleri silebilir.

4.2 Sınıf Diyagramı



C-1: User (Kullanıcı) Sınıfı

Sisteme giriş yapabilen tüm aktörlerin temel bilgilerini içeren ana sınıftır.

- **Özellikler:** `username`, `sifre`, `email`, `ad`, `soyad`.
- **Metotlar:** `login()` (Giriş yap), `logout()` (Çıkış yap).
- **İlişki:** `Ogrenci`, `Akademisyen` ve `Yonetici` sınıfları bu sınıftan türetilmiştir (Inheritance).

C-2: Ogrenci Sınıfı

Sistemdeki öğrenci kullanıcılarını temsil eder.

- **Özellikler:** `ogr_no` (Benzersiz öğrenci numarası), `bolum`, `sinif`, `gpa` (Genel not ortalaması).
- **Metotlar:**
 - `ders_kaydi_yap()`: Öğrencinin seçtiği dersleri onaya gönderir.
 - `ders_ve_not_goruntule()`: Alınan derslerin notlarını ve harf notlarını listeler.
 - `transkript_goruntule()` ve `transkript_indir_pdf()`: Akademik geçmişi görüntüler ve PDF olarak kaydeder.

C-3: Akademisyen Sınıfı

Eğitim süreçlerini ve notlandırmayı yöneten kullanıcıları temsil eder.

- **Özellikler:** `personel_id`, `unvan`.
- **Metotlar:**
 - `verilen_dersleri_listele()`: Hocanın o dönem sorumlu olduğu dersleri gösterir.
 - `ogrenci_listesi_goruntule()` ve `not_gir_guncelle()`: Derse kayıtlı öğrencilerin vize/final notlarını yönetir.
 - `ders_kaydi_onayla()`: Öğrencilerin ders kayıt taleplerini onaylar.

C-4: Yonetici Sınıfı

Sistemin idari yönetiminden sorumlu personeli temsil eder.

- **Özellikler:** `yonetici_id`.
- **Metotlar:**
 - `ogrenci_kaydi_excel(dosya: Any)`: Excel üzerinden toplu öğrenci kaydı yapar.
 - `akademisyen_kaydi_manuel()` ve `kullanici_yonetimi()`: Kullanıcı veritabanını yönetir.
 - `ders_olustur()`, `ders_guncelle()`, `ders_sil()`: Müfredattaki ders tanımlarını yönetir.

C-5: DonemDersi Sınıfı

Bir dersin belirli bir akademik yıl ve dönemdeki somut açılışını temsil eder.

- **Özellikler:** `dersin_hocasi`, `yil`, `donem`, `kontenjan`, `aktiflik_durumu`.
- **İlişki:** Bir `Ders` tanımına bağlıdır ve bir `Akademisyen` tarafından verilir.

C-6: DersKaydi Sınıfı

Öğrenci ile Dönem Dersi arasındaki ilişkiyi ve bu dersin başarı durumunu tutar.

- **Özellikler:** `vize_notu`, `final_notu`, `ortalama`, `harf_notu`, `onay_durumu`.
- **Metotlar:**
 - `ortalama_hesapla()`: Vize %40 ve Final %60 ağırlığına göre puan üretir.
 - `harf_notu_atama()`: Hesaplanan ortalamaya göre harf notu belirler.

4.3 Durum Diyagramı

[Bu bölümü doldurun.]

4.4 Aktivite Diyagramı

[Bu bölümü doldurun.]

4.5 Sıra (Sequence) Diyagramı

[Bu bölümü doldurun.]

4.6 Kullanıcı Arayüzü

[Bu bölümü doldurun.]

5. Referanslar

Söylev, Arda. Yazılım Mühendisliği Ders Slaytları, 2026.

[Bu bölüme kullandığınız referansları ekleyin.]